

Modelisation et Optimisation

TP1,2 et 3 fait par: *Marc Achraf Atef MORCOS*

1. Concept

Le problème du sac à dos 0–1 est un problème classique d’optimisation combinatoire.

Étant donné un ensemble d’objets caractérisés par une valeur et un poids, l’objectif est de maximiser la valeur totale tout en respectant une contrainte de capacité.

Dans ce travail, nous étudions plusieurs approches : méthode gloutonne, relaxation linéaire, programmation dynamique et techniques de réduction.

2. Implementation

1. Structures de données (`TP1Functions.h`)

`dataSet`

Cette structure contient une instance du sac à dos :

- `n` : nombre d’objets
- `b` : capacité du sac
- `c[i]` : valeur de l’objet i
- `a[i]` : poids de l’objet i

L’instance est donc représentée par deux tableaux d’entiers de taille `n` (`c[]` et `a[]`), ce qui permet un accès direct en $O(1)$ à chaque objet.

`utilite`

Cette structure sert au tri par utilité :

- `util` : utilité c_i/a_i

- `index` : l'indice original de l'objet dans `c[]` et `a[]`

On trie un tableau de `utilite` au lieu de permuter les tableaux `c[]/a[]`, ce qui évite de modifier les données originales

2. Lecture de l'instance (`read_TP1_instance`)

- lire `n` et `b` depuis le fichier
- allouer dynamiquement `c` et `a`
- lire les `n` couples `(c[i], a[i])`
- utilisation de `malloc(sizeof(int)*n)` pour adapter la taille aux instances

3. Calcul des utilités (`util_calc`)

Pour chaque objet i , on calcule :

$$u_i = c_i / a_i$$

Puis on stocke :

- `utility[i].util = u_i`
- `utility[i].index = i`

Complexité : $O(n)$

4. Tri décroissant par utilité (`sort` , `comp`)

Comparateur `comp`

Le comparateur ordonne les objets selon `util` décroissant :

- si $utilA < utilB \Rightarrow utilB$ avant $utilA$
- si $utilA > utilB \Rightarrow utilA$ avant $utilB$

Tri `qsort`

`qsort(utility, n, sizeof(utilite), comp)` trie le tableau.

Complexité : $O(n \log n)$

5. Initialisation (`init`)

Rôle :

- mettre `x_bar[i]=0` pour tous les objets
- initialiser la capacité restante `b_bar = b`

Ce tableau `x_bar` représente la solution :

- en Greedy : $x_i \in \{0, 1\}$
- en LP : $x_i \in [0, 1]$

Complexité : $O(n)$

6. Algorithme glouton (`KP_greedy`)

Principe

La méthode gloutonne sélectionne les objets selon le rapport utilité

$$c_j/a_j$$

Cette stratégie est optimale pour la version fractionnaire du problème, mais n'est pas garantie optimale pour la version 0–1.

Elle fournit cependant une solution réalisable rapide et souvent proche de l'optimum.

- calcul utilités + tri
- parcourir les objets triés
- ajouter un objet si son poids rentre dans la capacité restante

Pseudo-code :

- si `b_bar >= a[j]` alors `x_bar[j]=1` et `b_bar -= a[j]`

Valeur

On cumule la valeur :

$$z = \sum_{j=1}^n c_j x_j$$

Complexité : $O(n \log n)$

Résultat :

- solution réalisable

7. Relaxation linéaire (KP_LP)

Principe

La relaxation linéaire remplace la contrainte binaire

$$x_j \in \{0, 1\} \quad \text{par} \quad x_j \in [0, 1]$$

Le problème devient alors un programme linéaire résoluble efficacement.

donc c'est le même tri, mais on autorise une fraction :

$$x_j = \min \left(1, \frac{b_{\text{restant}}}{a_j} \right)$$

Donc :

- objets entièrement si possible ($x=1$)
- puis un objet fractionnaire ($0 < x < 1$)
- puis on arrête car la capacité devient 0

Valeur

$$\bar{z} = \sum_{j=1}^n c_j x_j$$

Complexité : $O(n \log n)$

Résultat :

- solution éventuellement non entière
- borne supérieure

8. Programmation Dynamique (DP)

Principe général

Contrairement aux méthodes gloutonne et relaxation linéaire, la programmation dynamique permet de résoudre exactement le problème du sac à dos 0-1.

L'idée est de construire progressivement la meilleure valeur possible pour chaque capacité intermédiaire :

$$z(y) = \text{meilleure valeur atteignable avec capacité } y$$

pour $y = 0, 1, \dots, b$.

Construction de la table DP

On commence par trier selon l'utilité.

On définit un tableau :

- $z[y]$: valeur maximale obtenue pour la capacité y

Pour chaque objet j , on met à jour la table selon la relation :

$$z_{new}(y) = \max(z_{old}(y), z_{old}(y - a_j) + c_j) \text{ si } y \geq a_j$$

Interprétation :

- soit on ne prend pas l'objet j ,
- soit on le prend, si la capacité restante le permet.

Implémentation dans le code (KP_PD)

La fonction : `KP_PD(DataSet *dptr , int *x_etoile)`

prend en entrée une instance du problème et retourne :

- la valeur optimale z^* ,

- le vecteur optimal x^* .

Tableaux utilisés

- $z[y]$: valeur courante
- $z_bar[y]$: copie temporaire pour garantir la contrainte 0–1
- $d[y]$: mémorise l'objet choisi pour reconstruire la solution

L'utilisation de z_bar permet d'éviter qu'un même objet soit pris plusieurs fois.

Reconstruction de la solution

Après le calcul de $z[b]$, on reconstruit la solution optimale en remontant la table :

1. On initialise tous les $x_j = 0$
2. On part de $y = b$
3. Tant que $z[y] > 0$:
 - si $z[y] = z[y - 1]$, on décrémente y
 - sinon, on sélectionne l'objet mémorisé dans $d[y]$
 - on met $x_j = 1$
 - on réduit la capacité $y = y - a_j$

On obtient ainsi une solution binaire optimale.

Complexité

La programmation dynamique comporte :

- une boucle sur les objets n
- une boucle sur les capacités b

La complexité temporelle est donc : $O(n \cdot b)$

La complexité mémoire est : $O(b)$

9. Génération d'instances

On a créé un programme dédié `TP1GenInstance.c` qui produit des fichiers `.csv` au format demandé : première ligne `n,b` puis `n` lignes `cj, aj` (valeur, poids).

Le programme prend en paramètres le nom du fichier, le nombre d'objets `n` et la capacité `b`, avec les contraintes du sujet spécifiées telles que: $1 \leq n \leq 100$, $1 \leq b \leq 1000$)

Les coefficients sont générés aléatoirement avec `rand()` initialisé par `srand(time(NULL))`.

Chaque instance est donc reproductible en relançant la génération (ou en fixant une graine).

Paramètres utilisés

- capacité : `b` donnée en entrée
- poids : $a_j \in [1, b]$
- valeurs : $c_j \in [1, 1000]$

Cette génération permet de produire un large ensemble d'instances pour analyser le comportement moyen des algorithmes.

10. Mesure expérimentale du temps

Pour étudier la complexité temporelle, nous avons implémenté un programme `TP1CourbeTemps.c` qui mesure le temps moyen d'exécution de chaque algorithme :

- Greedy
- Relaxation LP
- DP

Le temps est mesuré avec une fonction `now_sec()` basée sur `gettimeofday`.

Afin de réduire le bruit, chaque point de courbe est une moyenne :

- sur `R` répétitions par instance,
- et sur `INST` instances aléatoires différentes.

Les résultats sont enregistrés au format CSV :

- `bench_nfixed.csv` : n fixé, b variable

- `bench_bfixed.csv` : b fixé, n variable

(a) n fixé, b variable

On fixe $n=100$ et on fait varier b .

Théoriquement, **Greedy** et **LP** ont une complexité $O(n \log n)$, donc indépendante de b ,

donc leurs temps restent constantes.

Tandis que puisque **DP** a une complexité $O(n \cdot b)$ et dépend donc de n et b , donc son temps augmente linéairement avec b

(b) b fixé, n variable

On fixe $b=500$ et on fait varier n .

Le coût dominant est le tri $O(n \log n)$, donc Greedy et LP doivent croître de façon proche de $n \log n$.

Tandis que DP croît linéairement avec n .

Ainsi, pour des grandes valeurs de n , DP peut devenir plus coûteux que Greedy et LP malgré sa solution optimale.

Tracer les courbes de temps

Après la génération des fichiers `bench_nfixed.csv` et `bench_bfixed.csv`, un script Python a été utilisé pour tracer les courbes de complexité.

Ce script :

- lit les fichiers CSV générés par le programme de benchmark,
- extrait les colonnes correspondant aux temps moyens d'exécution,
- trace les courbes en fonction de n ou b
- permet une visualisation comparative des trois algorithmes (Greedy, LP, DP).

11. Réduction du problème

Principe général

L'objectif de la réduction est de diminuer la taille du problème du sac à dos avant d'appliquer un algorithme exact (programmation dynamique).

L'idée repose sur la comparaison entre :

- une **borne inférieure** obtenue par la solution gloutonne $z(\tilde{x})$,
- une **borne supérieure** donnée par la relaxation linéaire $\bar{z}$.

Si certaines variables ne peuvent pas améliorer suffisamment la solution, elles peuvent être **fixées définitivement** à 0 ou 1.

Cela permet de construire un problème réduit plus petit.

Étapes de la méthode

1. Calcul des bornes

On commence par calculer :

- solution gloutonne $\rightarrow z_x$
- solution LP $\rightarrow \bar{z}$

avec :

$$z_x \leq z^* \leq \bar{z}$$

où z^* est la valeur optimale.

2. Identification de l'objet fractionnaire

Dans la solution LP :

- les premiers objets sont pris entièrement,
- un seul objet peut être fractionnaire,
- les suivants sont exclus.

On note p la position de cet objet.

Son ratio :

$$\lambda = c_p/a_p$$

sert de référence pour la réduction.

3. Calcul du coût réduit

Pour chaque objet j , on calcule :

$$\bar{c}_j = |c_j - \lambda a_j|$$

Ce terme mesure l'écart entre l'objet et l'objet fractionnaire LP.

4. Critère de fixation

On compare ce coût réduit à l'écart entre les bornes :

$$\bar{z} - z_x$$

- si $\bar{c}_j \geq \bar{z} - z_x$

alors la variable peut être fixée :

- objets avant $p \rightarrow x_j = 1$
- objets après $p \rightarrow x_j = 0$

Ces objets ne peuvent pas changer l'optimalité.

5. Construction du problème réduit

Les variables restantes (non fixées) forment un nouveau problème :

$$PR : \max \sum c_j x_j$$

$$\sum a_j x_j \leq b_R$$

où b_R est la capacité restante après fixation des objets à 1.

Ce problème contient seulement m variables avec :

$$m < n$$

6. Résolution finale

Le problème réduit est ensuite résolu par programmation dynamique : `KP_PD( &red , x_red )`

La solution finale est obtenue en combinant :

- variables fixées,
- solution du problème réduit.

Implémentation (fonction KP_VP)

La fonction `KP_VP` réalise successivement :

1. calcul Greedy et LP,
2. détection de l'objet fractionnaire,
3. fixation des variables (`x_bar`),
4. construction de l'instance réduite,
5. appel de la programmation dynamique,
6. reconstruction de la solution globale.

Le tableau :

```
x_bar[j] = -1  → variable libre
x_bar[j] = 0   → fixée à 0
x_bar[j] = 1   → fixée à 1
```

permet de distinguer les états.

12. Analyse expérimentale de la réduction

L'intérêt de la réduction est évalué selon trois critères.

12.1 Proportion de problèmes réduits

On mesure la proportion d'instances pour lesquelles :

$$m < n$$

c'est-à-dire lorsque la réduction élimine au moins une variable.

Dans le code :

```
static int total = 0;  
static int reduced = 0;
```

La proportion affichée est :

$$reduced/total$$

Une proportion élevée indique que la méthode est efficace.

12.2 Pourcentage de variables fixées

On distingue :

- variables fixées à 1,
- variables fixées à 0,
- variables libres.

Le pourcentage est calculé par :

$$(variables\ fixées/n) * 100$$

Cette mesure indique la réduction effective de la taille du problème.

Plus ce pourcentage est grand, plus le gain potentiel est important.

12.3 Gain de temps de résolution

On compare :

- temps DP original :

$$T_{DP}$$

- temps total avec réduction :

$$T_{VP}$$

Le gain relatif est :

$$Gain = (T_{DP} - T_{VP}) / T_{DP} \times 100$$

et le facteur d'accélération :

$$Speedup = T_{DP} / T_{VP}$$

Interprétation

Expérimentalement, on observe que :

- de nombreuses variables sont fixées avant la résolution exacte,
- la taille du problème traité par DP diminue,
- le temps d'exécution total est réduit.

La réduction agit donc comme une **phase de pré-traitement** améliorant significativement la résolution exacte.

13. Conclusion sur la réduction

La technique de variable fixing combine efficacement :

- rapidité des méthodes approchées (Greedy, LP),
- exactitude de la programmation dynamique.

Elle permet :

- de réduire la dimension du problème,
- d'accélérer la résolution,
- tout en conservant l'optimalité.

Cette approche illustre l'intérêt de combiner bornes et méthodes exactes en optimisation combinatoire.

14. Programme principal (**TP1.c**) :

1. lire le fichier instance (**F** optionnel) ou plusieurs instances
2. charger l'instance dans **dataSet data**
3. exécuter :
 - **KP_greedy(&data)**
 - **KP_LP(&data)**
 - **KP_PD(&data, x_dp)**
 - **KP_VP(&data)** son analyse locale et l'analyse globale

14. Resultats :

```
==== instance2.csv ====
Instance file read, we have capacity 11 and there is 4 items of values/weights:
12,5
8,4
2,1
5,3

Valeur totale du sac a dos utilisant la methode gloutonne : 22
Valeur optimale du sac a dos utilisant LP : 23.67
Optimal value with DP: 22
Chosen items (x*): 1 1 1 0
Valeur totale du sac a dos utilisant la methode gloutonne : 22
Valeur optimale du sac a dos utilisant LP : 23.67
Optimal value with DP: 10
Chosen items (x*): 1 1 0
[VP] x*: 1 1 1 0

==== ANALYSE Locale ====
fixed=1/4 (25.00%), fixed0=0 (0.00%), fixed1=1 (25.00%), free=3
Temps DP original = 0.000009 secondes
Temps total VP = 0.000026 secondes
Gain = -186.84% | Speedup = 0.35x

Instance file read, we have capacity 53 and there is 12 items of values/weights:
14,7
13,7
15,10
10,7
8,6
13,10
10,8
10,9
5,5
7,10
7,12
6,13

Valeur totale du sac a dos utilisant la methode gloutonne : 78
Valeur optimale du sac a dos utilisant LP : 80.50
Optimal value with DP: 78
Chosen items (x*): 1 1 1 1 1 1 0 0 1 0 0 0
Valeur totale du sac a dos utilisant la methode gloutonne : 78
Valeur optimale du sac a dos utilisant LP : 80.50
Optimal value with DP: 36
Chosen items (x*): 1 1 1 0 0 1
[VP] x*: 1 1 1 1 1 1 0 0 1 0 0 0

==== ANALYSE Locale ====
fixed=6/12 (50.00%), fixed0=3 (25.00%), fixed1=3 (25.00%), free=6
Temps DP original = 0.000008 secondes
Temps total VP = 0.000026 secondes
Gain = -230.30% | Speedup = 0.30x
```

la resultat de l'instance 2 qui est utilise comme
exemple en TP1 et TP2:

la resultat de l'instance 3 qui est utilise
comme exemple dans TP3

```
==== instance9.csv ====
Instance file read, we have capacity 10 and there is 5 items of values/weights:
993,6
649,10
201,4
496,6
173,3

Valeur totale du sac a dos utilisant la methode gloutonne : 1166
Valeur optimale du sac a dos utilisant LP : 1323.67
Optimal value with DP: 1194
Chosen items (x*): 1 0 1 0 0
Valeur totale du sac a dos utilisant la methode gloutonne : 1166
Valeur optimale du sac a dos utilisant LP : 1323.67
Optimal value with DP: 201
Chosen items (x*): 1 0 0
[VP] x*: 1 0 1 0 0

==== ANALYSE Locale ====
fixed=2/5 (40.00%), fixed0=1 (20.00%), fixed1=1 (20.00%), free=3
Temps DP original = 0.000007 secondes
Temps total VP = 0.000027 secondes
Gain = -289.66% | Speedup = 0.26x
```

```
==== instance8.csv ====
Instance file read, we have capacity 2 and there is 1 items of values/weights:
310,2

Valeur totale du sac a dos utilisant la methode gloutonne : 310
Valeur optimale du sac a dos utilisant LP : 310.00
Optimal value with DP: 310
Chosen items (x*): 1
Valeur totale du sac a dos utilisant la methode gloutonne : 310
Valeur optimale du sac a dos utilisant LP : 310.00
[VP] LP integral (no fractional item). Nothing to fix.
```

exemple d'une instance generee qui n'etait pas reduite.

exemple d'une instance generee qui
etait reduite.

2. Analyse des valeurs

On observe que dans plusieurs instances :

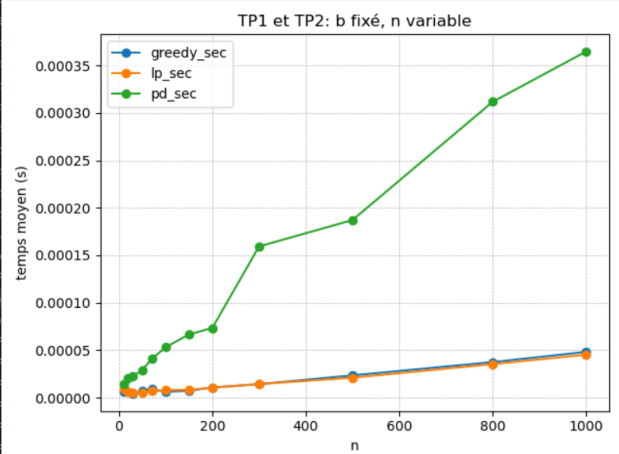
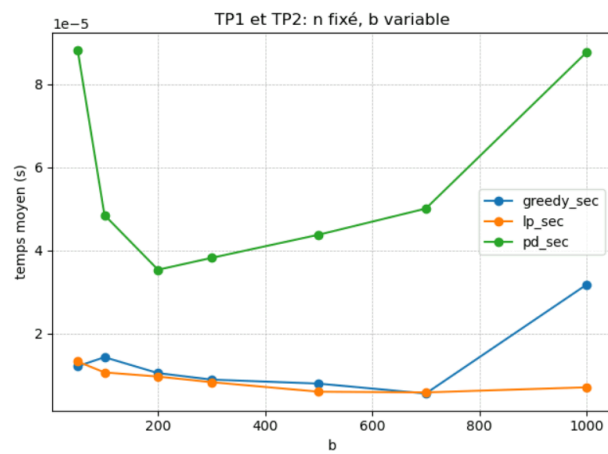
- Greedy = DP (solution optimale atteinte)
- $LP \geq DP$ toujours (borne supérieure respectée)

3. Analyse de la reduction

- Le pourcentage de variables fixées augmente avec n
- Le gain devient positif pour grandes instances
- Sur petites instances, la réduction peut être plus coûteuse que DP (overhead)
- La proportion de problèmes réduits est élevée (ex: 88.89%)

```
==== ANALYSE GLOBALE ====
Proportion de problemes reduits = 8/9 (88.89%)
```

3. Analyse des courbes de temps



- Greedy et LP sont presque constants quand b varie
- DP augmente linéairement avec b
- Quand n augmente, DP croît plus vite que Greedy/LP